

Recurrence Relation:

How to decrease?
How fast are we converging?

$$T(n) = aT(n-b) + f(n)$$

Decomposition

Composition

No. of recursive calls in one call.

a, b : constants

$f(n)$ is any mathematical function

Recursive Functions.

Void Test (int n)

{

if (n > 0)

1

{

cout << n; ← ①

1

Test (n-1),

$T(n-1)$

}

}

$$T(n) = T(n-1) + 1 \quad \text{--- } M$$

put $n = n-1$ in M

$$T(n-1) = T(n-2) + 1$$

put $T(n-1)$ in M

$$T(n) = T(n-2) + 2$$

--- (i)

put $n = n-2$ in M

$$T(n-2) = T(n-3) + 1$$

put $T(n-2)$ in (i)

$$T(n) = T(n-3) + 3$$

• kth iteration

$$T(n) = T(n-k) + k$$

Base case when

$$n-k=0$$

$$k=n$$

$$T(n) = T(n-n) + n$$

$$= T(0) + n$$

$$= 1 + n$$

$$= O(n)$$

void Test(int n) {

if (n > 0) { _____ 1

for (int i = 0; i < n; i++) { _____ n + 1

} printf(n); _____ n

} Test(n-1); _____ T(n-1)

}

$$T(n) = T(n-1) + n \quad \text{--- } 1$$

put $n = n-1$ in M

$$T(n-1) = T(n-2) + n-1$$

put $T(n-1)$ in M

$$T(n) = T(n-2) + (n-1) + n \quad \text{--- } 1$$

put $n = n-2$ in M

$$T(n-2) = T(n-3) + n-2$$

put $T(n-2)$ in (i)

$$T(n) = T(n-3) + (n-2) + (n-1) + n \quad \text{--- } 1$$

⋮
⋮
⋮
kth iteration

$$T(n) = T(n-k) + (n-(k-1)) + \dots + (n-2) + (n-1) + n$$

Base case reached when

$$n-k=0$$

$$k=n$$

$$\begin{aligned} T(n) &= T(n-n) + (n-(n-1)) + \dots + (n-2) + (n-1) + n \\ &= T(0) + 1 + \dots + (n-2) + (n-1) + n \\ &= \frac{1 + n(n+1)}{2} \end{aligned}$$

$$= O(n^2)$$

void Test(int n) {

if (n > 0) { _____ 1

for (int i = 0; i < n; i++) { _____ n+1

Test(n-1); _____ n

}

$$T(n) = nT(n-1) + n \quad \text{--- M}$$

put $n=n-1$ in M

$$T(n-1) = (n-1)T(n-2) + (n-1)$$

put $T(n-1)$ in M

$$T(n) = n(n-1)T(n-2) + n(n-1) + n \quad \text{--- (i)}$$

put $n=n-2$ in M

$$T(n-2) = (n-2)T(n-3) + (n-2)$$

put $T(n-2)$ in (i)

$$T(n) = n(n-1)(n-2)T(n-3) + n(n-1)(n-2) + n(n-1) + n$$

• kth iteration

b

$$T(n) = n(n-1)(n-2)\dots(n-k+1)T(n-k) + n(n-1)(n-2)\dots(n-k+1)$$

$$+ \dots + n(n-1) + n$$

Base case reached when

$$n-k=0$$

$$k=n$$

$$T(n) = n(n-1)(n-2) \dots (n-(n-1))T(n-n) + n(n-1)(n-2) \dots (n-(n-1)) + \dots + n(n-1) + n$$

$$\begin{aligned} T(n) &= n(n-1)(n-2) \dots 1T(0) + n(n-1)(n-2) \dots 1 + \dots + n \\ &= n(n-1)(n-2) \dots 1 + n(n-1)(n-2) \dots 1 + \dots + n(n-1) + n \\ &= n! + n! + \dots + n(n-1) + n \\ &= O(n!) \end{aligned}$$

void fib (int n) {

if (n == 0 or n == 1) { — 1

return n; — 1

} return fib(n-1) + fib(n-2); — $T(n-1)$
 $T(n-1)$ $T(n-2)$

↳ converges faster.
 So, for simplicity we say $2T(n-1)$ as this upper bound.

$$T(n) = 2T(n-1) + 1 \text{ — } M$$

put $n = n-1$ in M
 $T(n-1) = 2T(n-2) + 1$

put $T(n-1)$ in M ;
 $T(n) = 2^2T(n-2) + 2 + 1 \text{ — } (i)$

put $n = n-2$ in M
 $T(n-2) = 2T(n-3) + 1$

put $T(n-2)$ in $\mathcal{B}$,

$$T(n) = 2^3 T(n-3) + 2^2 + 2 + 1$$

• k th iteration
•

$$T(n) = 2^k T(n-k) + 2^{k-1} + \dots + 2^2 + 2 + 1$$

Base case reached when

$$n-k=0$$

$$k=n$$

$$T(n) = 2^n T(n-n) + 2^{n-1} + \dots + 2^2 + 2 + 1$$

$$= 2^n T(0) + 2^{n-1} + \dots + 2^2 + 2 + 1$$

$$= 2^n + 2^{n-1} + \dots + 2^2 + 2 + 1$$

$$= 2^{n+1} - 1$$

$$= 2 \cdot 2^n - 1$$

$$O(2^n)$$